

CLI Commands

```
lambda           # Start REPL (cmds: quit, help)
lambda script.ls  # Eval a functional script
lambda run script.ls  # Run a procedural script
lambda --help     # Show help
```

The dev build is ./lambda.exe; release bundles ship it as

lambda.

Documents:

```
lambda validate file.json -s schema.ls      # Against a schema
lambda validate page.html                   # Built-in schema (html, md...)
lambda convert doc.md -t html -o doc.html
lambda render page.html -o page.pdf          # To svg/pdf/png/jpg
lambda view page.html                       # Open the interactive viewer
lambda edit notes.md                        # Edit Markdown, HTML or SVG
```

Type System

Scalar Types:

null bool int integer float	Primitives (integer is arbitrary precision: 42n)
decimal	Primitives
string symbol binary datetime	Primitives
path	
i8 i16 i32 i64	Sized signed integers
u8 u16 u32 u64	Sized unsigned integers
f16 f32 f64	Sized floats
any none	Top type (every value); empty type (no value, not even null)

Container Types:

range, 1 to 10	Range (inclusive both ends; also “a” to “z”)
array, [123, true]	Array of values
map, {key: 'symbol'}	Map
element, <div class: bold, "text"> >	Element

Type Operators:

int string	Union type
int & number	Intersection
int ! string	Exclusion
1 2	Literal union (enum) type — in an expression too
1 & 2	none , the empty type: admits nothing
int?	Optional (int null)
int*	Zero or more
int+	One or more
int[]	Array of ints (same as int*)
int[5]	Array of exactly 5 ints
[int*]	Bracket form: array of 0+ ints
[int+]	Bracket form: array of 1+ ints
fn (a: int, b: string) bool	Function type
fn () int	Function type with no parameters
{a: int, b: bool}	Map type
<div id: symbol, >	Element type

Type Declarations:

Lambda Script Cheatsheet

```
type User = {name: string, age:  Map type alias
  int}
type Point = (float, float)      Tuple type
type Result = int | error        Union type
```

Object Types

Definition:

```
type Point { x: float, y: float } // Fields only
type Counter {
  value: int = 0, // Default value
  fn double() => value * 2 // Functional
  fn add(n: int) => value + n
  pn inc() { value = value + 1 } // Procedural
}
type Circle : Point { radius: float } // Inheritance
```

Literals & Access:

```
let p = <Point x: 1.0, y: 2.0> // Object literal
let c = <Counter>              // All defaults
p.x                            // Field access
c.double()                     // Method call
<Point *:p, x: 5.0>            // Copy and override
```

Type Checking (nominal only):

```
p is Point      true
p is object     true
p is map        true (objects are map-compatible)
{x: 1} is Point false (plain maps don't match)
```

Constraints:

```
type User {
  name: string that (len(-) > 1), // Field constraint
  age: int that (0 <= ~ <= 150),
  that (~.name != "admin")        // Object constraint
}
type Age = int that (0 <= ~ <= 150)
```

Self reference ~:

```
type Vec {
  x: float, y: float,
  fn len() => math.sqrt(x**2 + y**2)
  fn scale(f) => <Vec *:~, x: ~.x*f, y: ~.y*f> // ~ = self
}
```

Literals

Numbers:

42	Integer
42n	Arbitrary precision integer
3.14	Float
1.5e-10	Scientific notation
123.45m	Decimal
1e-3m	Decimal (negative exponent)
inf nan	Special values
0xFF	Hexadecimal int literal

Sized numeric literals (postfix suffix):

Oct 2026 - v0.3 (alpha)

42i8 -128i8 1000i16 100000i32	Signed
100i64	
255u8 60000u16 3000000000u32	Unsigned
1000u64	
0.5f16 3.14f32 2.7f64	Float

Strings & Symbols:

"hello"	String
"multi-line string"	Multi-line string — a literal newline may appear inside the quotes
'symbol'	Symbol
symbol	Unquoted symbol
'name' == "name"	false — symbol is not string
"" == null	false — empty string is a string value
not ""	true — empty string is falsy

'' is invalid: empty symbols do not exist.

Binary:

b'\xDEADBEEF'	Hex binary
b'\x64QUVGRw=='	Base64 binary

DateTime:

t'2025-01-01T14:30:00Z'	DateTime
t'2025-04-26' is date	Sub-type: date
t'10:30:00' is time	Sub-type: time

Member properties:

dt.date dt.year dt.month dt.day	Date parts
dt.time dt.hour dt.minute	Time parts
dt.second dt.millisecond	
dt.weekday dt.yearday dt.week	Derived
dt.quarter	
dt.unix dt.timezone	Zone / epoch
dt.utc_offset dt.utc dt.local	

Formatting:

dt.format("YYYY-MM-DD")	Pattern
dt.format('iso')	Named format

Constructors:

datetime() today() justnow()	Current date/time
datetime("2025-04-26T10:30")	From an ISO string / Unix
datetime(1745643645123)	milliseconds
date(2025, 4, 26)	Date / time from parts
time(10, 30, 45)	

Collections:

[1, 2, 3]	Array
{a: 1, b: 2}	Map
{[key]: value}	Map field whose string/symbol key is computed at run time
<div id: "main">	Element
<input [name]: value>	Element attribute whose string/symbol name is computed at run time

Indexing & Slicing:

arr[0]	First element
arr.0	Alt. syntax for const index
arr[1 to 3]	Slice (indices 1, 2, 3)
arr[[1, 3, 9]]	Index array: [arr[1], arr[3], null]
arr[[true, false, true]]	Mask: the marked positions
map.key	Map field access
map["key"]	Map field by string
"hello"[1 to 3]	“ell” — string slicing
'hello'[1 to 3]	‘ell’ — symbol slicing
"café"[2 to 3]	“fé” — UTF-8 aware

Namespaces (via import with bare URI):

import svg: 'http://www.w3.org/2000/svg'	
import xlink: 'http://www.w3.org/1999/xlink'	
<svg.rect svg.width: 100>	Namespaced tag & attribute — desugars to <svg.rect svg: {width: 100}>
elem.svg.width	Chained sub-map access
elem.svg	{width: 100} sub-map
svg.rect	Qualified symbol
symbol("href", 'xlink_url')	Namespaced symbol

Variables & Declarations

Let Expressions (immutable):

(let x = 5, x * 2)	Single binding
(let a = 1, let b = 2, a + b)	Multiple bindings — repeat let

Let Statements (immutable):

let x = 42	Immutable binding
let y : int = 100	With type annotation
let a: i8 = 42i8	Sized type annotation
let b: f32 = 3.14f32	Sized float annotation
let a = 1, b = 2	Multiple bindings
x = 10	ERROR E211: cannot reassign let binding

Var Statements (mutable, pn only):

var x = 0	Mutable variable
var y: int = 42	With type annotation
var a: int[] = [1, 2, 3]	Array of ints
var b: float[] = [0.1, 0.2]	Array of floats
x = x + 1	OK: reassignment
x = "hello"	OK: type widening (int → string)
y = "oops"	ERROR E201: annotated type enforced

Operators

Arithmetic:

Operator	Meaning
+	addition
-	subtraction
*	multiplication
/	division
div	integer division
%	modulo
**	exponentiation

Spread *:

Given	Expression	Result
let a = [1, 2, 3]	(*a, *[10, 20])	(1, 2, 3, 10, 20)

Comparison:

Operator	Meaning
==	equal
!=	not equal
<	less than
<=	less or equal
>	greater than
>=	greater or equal

== performs structural deep equality on all types:

[1, 2] == [1, 2]	true (array)
{a:1, b:2} == {b:2, a:1}	true (map, order-independent)
[1] == [1.0]	true (numeric promotion)
(1 to 3) == [1, 2, 3]	true (cross-type sequence)

Logical:

Operator	Meaning
and	logical and
or	logical or
not	logical not

Type & Set:

Operator	Meaning
is	type check
in	membership
to	range
	union
&	intersection
!	exclusion

Vector Arithmetic:

Expression	Result	
1 + [2, 3]	[3, 4]	scalar broadcast
[1, 2] * 2	[2, 4]	scalar broadcast
[1, 2] + [3, 4]	[4, 6]	element-wise
[1, 2, 3] eq 2	[false, true, false]	element-wise comparison:
		eq ne lt le gt ge
		(S10.2.2)
[10, 20, 30][[1, 2, 3] gt 1]	[20, 30]	mask indexing;
		sum(mask) counts

Use ++ for list/array concat: [1,2] ++ [3,4] = [1,2,3,4].

Pipe Expressions

Pipe |> with current item ~:

[1,2,3] > ~ * 2	[2,4,6] - map over items
[1,2,3] > sum	6 - aggregate (no ~)
users > ~.age	[12,20,62] - extract field
['a', 'b'] > {i:~key, v:~}	~key = index/key

Filter with |: (the body must mention ~):

[1,2,3,4,5] : (~ > 3)	[4,5]
users : ~.age >= 18 > ~.name	filter then map
[1,2,3] > ~ ** 2 : (~ > 3) > sum	13 (4+9)

Self-qualifier with that (one value, never walked):

5 that ~ > 3	5
5 that ~ > 9	null
[1,2,3] that len(~) > 2	[1,2,3] - the whole array

Pipe and filter results in array literals: an array result is one item

[1, [2,3] > ~, 4, 5]	[1, [2, 3], 4, 5]
[0, *([1,2,3] > ~ * 10), 99]	[0, 10, 20, 30, 99] - * splices
[1, [3,5,7] : (~ > 4), 9]	[1, [5, 7], 9]

Query Expressions

Results in document order (depth-first, pre-order).

Query ? - search attributes + all descendants:

html?	all at any depth
html?<div class: string>	<div> with class attr
data?int	all int values in tree
data?(int string)	all int or string values
data?{name: string}	maps with string 'name'
data?{status: "ok"}	maps where status == “ok”

Self-inclusive query .? — search self + attributes + all descendants:

div.?<div>	includes div itself if it matches
el.?int	self + all int values in subtree
42.?int	42 — a lone match is the value itself

Child-level query [τ] — search direct attributes + children only:

[1, "a", 3, true][int]	[1, 3] — int items
{name: "Alice", age: 30}[string]	["Alice"]
]	
el[element]	direct child elements only
el[string]	attr values + text children

Control Flow

If Expressions (parenthesized condition, else required):

if (x > 0) "positive" else "non -positive"	Simple
if (score >= 90) "A" else if (score >= 80) "B" else "C"	Chained
if (x > 0) "pos" else { "neg" }	Block else

If Statements (block body, else optional):

if x > 0 { "positive" }	No else
if condition { something() }	Both blocks
else { otherThing() }	
if x > 0 { compute() } else "default"	Expression else

Both forms share the same else syntax: else expr,

else { stam }, Or else if

Match Expressions:

// Type patterns or Literal		
match value {		
case 200: "OK"	// case literal	
case string: "text"	// case type	

```
case int | float: "number" // case type union
case Circle:              // case object type
    3.14 * ~.r ** 2
    default: "other"
}

// String pattern arms (full-match semantics)
type digits = \(\d+)
type alpha = \(\a+)
match input {
    case digits: "number"      // case named pattern
    case alpha: "word"
    default: "other"
}
```

For Expressions: produce spreadable lists

Expression	Result
for (x in [1, 2, 3])	[2, 4, 6]
x * 2	
for (i in 1 to 5) i	[1, 4, 9, 16, 25]
* i	
for (x in data) if (x > 0) x else 0	conditional body
[for (i in 1 to 2) for (j in 1 to 2) i* j]	nested for-expressions flatten
[for (x in []) x]	empty for produces spreadable null, which is skipped
for (k at {a: 1, b: 2}) k	iterate a map by key
for (k, v at {a: 1, b: 2}) k ++ v	key and value (S8.1.3)

For Expression Clauses: let, where, group by, order by, limit, offset

```
for (x in data where x > 0) x // filter
for (x in data, let sq = x*x) sq // let binding
for (x in [3,1,2] order by x) x // (1,2,3)
for (x in [3,1,2] order by x desc) x // (3,2,1)
for (x in data limit 5 offset 10) x // pagination
for (x in data, let y=x*2
    where y>5 order by y desc limit 3) y
// g is an <group keys as attrs, members as children>
for (x in sales group by x.region into g)
    {region: g.region, n: len(content(g))}
```

Joins (on): relate comma sources; ? = left join

```
for (o in orders, c in customers on o.cust_id == c.id)
    {id: o.id, name: c.name} // inner equi-join
for (o in orders, c? in customers on o.cust_id == c.id)
    {id: o.id, name: c} // left join (c=null on miss)
```

For Statements:

```
for item in collection { transform(item) }
```

Procedural Control (in pn):

```
var x = 0 // Mutable variable
while (c) { break; continue; } // Loop and its jumps
return x }
```

Assignment Targets (in pn):

```
x = 10 // Variable reassignment (var only)
arr[i] = val // Array element reassignment
obj.field = val // Map field reassignment
elem.attr = val // Element attribute reassignment
elem[i] = val // Element child reassignment
```

Functions

Function Declaration:

```
fn add(a: int, b: int) int { a + b } // Statement body
fn multiply(x: int, y: int) => x * y // Expression body
let square = (x) => x * x // Anonymous function
let log = pn (msg) => { print(msg) } // Anonymous procedure (body always braced)
pn f(n) { var x = 0; while (x < n) { x = x + 1 }; x } // Procedural function
function each(f: function, xs) => for (x in xs) f(x) // Pure iff f is (S12.1.4v3)
pn advance(pos: float[], vel: float[], n: int) { ... } // Array parameters
```

Advanced Features:

```
fn f(x?:int) // optional param
fn f(x=10) // default param value
fn f(...) // variadic args
f(b:2, a:1) // named param call
fn outer(n) { fn inner(x)=>x+n; inner } // closure
```

Concurrency (pn only)

```
pn worker() { return receive()^ }
pn job(n) { sleep(n)^; return n }

pn main() {
    let h = start(worker) // start a child
    send(h, "job")^ // bounded FIFO mailbox
    print(wait(h)^) // wait for the T'E result

    let j = start(job, [5], {mode: 'task'})
    let k = start(job, [1])
    wait(j, timeout: 10)^ // timeout does not cancel j
    select(j, k, timeout: 100)^ // first completed handle
    sleep(5)^ // shared libuv timer
    self() // current handle
    cancel(k) // idempotent cancellation
    request
    io.read("file.txt")^ // async local file read
}
```

No `async/await`. Normal block exit joins children; error exit cancels then joins. A `start` operand may not capture an outer `var` by reference—copy to `let` or use messages.

String and Symbol Patterns

Define named patterns for string and symbol validation and matching. Uses regex-like syntax integrated into the type system.

```
type digits = \(\d+) // one or more digits
type email = \(\w+ "@" \w+ "." \a {2,6}) // email-like
type ws = \(\s+) // whitespace
type keyword = 'if' | 'else' | 'for' // symbol literal union
type SymIdent = \symbol(\a \w*) // symbol pattern
```

`\(...)` matches strings, `\symbol(...)` matches symbols. Character classes inside `\(...)`: `d` digit, `w` word, `s` whitespace, `a` alpha, `.` any char, `...` any string. Negation: `!` matches one character outside a set of single characters. Quantifiers: `?`, `+`, `*`, `{n}`, `{n,m}`, `{n+}`.

Inline (unnamed) patterns work anywhere a type does:

```
"abc" is email // false
"123" is digits // true
'foo' is SymIdent // true
"foo" is SymIdent // false — string value, symbol pattern
"abc" is \(\a+) // true
fn f(x: \(\d+)) => x // Parameter annotation
match s { case \(\d+): "num" // Match arm
default: "other" }
```

System Functions

Type:

```
int(v) i8(v) i16(v) i32(v) i64(v) u8(v) u16(v) u32(v) u64(v)
f32(v) float(v) decimal(v) string(v) symbol(v) binary(v) type(v)
name(v) len(v) count(v)
```

`type()` returns the canonical numeric name: `type(42i8) → "i8"`, `type(3.14f32) → "f32"`, `type(1.0f64) → "float"`

Math:

```
math.pi math.e abs(x) sign(x) min(a,b) max(a,b) round(x) floor(x)
ceil(x) math.trunc(x) math.sqrt(x) math.cbrt(x) math.hypot(x,y)
math.pow(b,e) math.log(x) math.log2(x) math.log10(x)
math.log1p(x) math.exp(x) math.exp2(x) math.expm1(x) math.sin(x)
math.cos(x) math.tan(x) math.asin(x) math.acos(x) math.atan(x)
math.atan2(y,x) math.sinh(x) math.cosh(x) math.tanh(x)
math.asinh(x) math.acosh(x) math.atanh(x) math.random(seed) → [value, new_seed]
```

Stats:

```
sum(v) avg(v) math.mean(v) math.median(v) math.variance(v)
math.deviation(v) math.quantile(v,p) math.prod(v)
```

Date/Time:

```
datetime() today() now() justnow() date(dt) time(dt)
```

Range:

`s` to `e` creates a range from `s` to `e` (inclusive both ends). Bounds are exact integers or single-codepoint strings. `range(s,e,step)` creates a range with custom step (exclusive end).

```
1 to 5                                [1, 2, 3, 4, 5]
"a" to "e"                            ["a", "b", "c", "d", "e"] — character
range(0, 10, 2)                       [0, 2, 4, 6, 8]
```

String:

```
join(strs,sep) normalize(str) ord(str) chr(int)
replace(str, pattern_or_string, replacement)
split(str, pattern_or_string) find(str, pattern_or_string) →
[{value, index}, ...]
```

Collection:

```
slice(v,i) slice(v,i,j) all(v) any(v) reverse(v) sort(v)
unique(v) intersect(a,b) except(a,b) take(v,n) drop(v,n) zip(a,b)
fill(n,x) range(s,e,step) reduce(v,f) contains(v,x) index_of(v,x)
map and filter are the pipes |> and |:
```

Vector:

```
math.dot(a,b) math.norm(v) math.cumsum(v) math.cumprod(v)
argmin(v) argmax(v) clip(v,lo,hi) shape(m) reshape(m,s)
transpose(m) flatten(m) matmul(a,b)

I/O:

input(file,fmt) format(data,fmt) print(v) output(data,file)
fetch(url,opts) cmd(c,args) error(msg) varg()
```

Input/Output Formats

Supported Input Types: json, xml, html, yaml, toml, ini, properties, csv, markdown, rst, asciidoc, wiki, org, textile, man, latex, typst, mark, rtf, pdf, eml, ics, vcf, css, math, graph, text

```
input("path/file.md", 'markdown')^ // Input Markdown
```

Input with Flavors: e.g. math flavors: latex, typst, ascii

```
input("math.txt", {'type':'math', 'flavor':'ascii'})^
```

Output Formatting: mark, json, yaml, toml, ini, properties, xml, html, markdown, rst, org, wiki, textile, latex, jsx, mdx, css, text, math (latex/ascii flavors), graph (dot/mermaid/d2)

```
format(data, 'yaml') // Format as YAML
```

When no specified output format:

- String data: raw text (no formatting)
- Binary data: raw binary data
- Other data: Lambda/Mark format

Modules, Imports & Exports

Import Syntax:

```
import module_name           Built-in module
import .relative_module      Relative to script's dir
import .path.to.module       Nested relative import
import alias: .module        Import with alias
```

Export Declarations:

```
pub let PI = 3.14159           // Export variable
pub fn square(x) => x * x       // Export function
pub pn log(msg) { print(msg) } // Export procedure
pub type Score = int           // Export type alias
```

```
pub type Counter {              // Export object type
  value: int = 0,
  fn double() => value * 2
}
pub let data = input("f") ^ { {} } // Export, handling err
```

Module Usage:

```
// no-run: shows two files in one block
// In math_utils.ls:
pub let PI = 3.14159
pub type Vec2 { x: float, y: float, fn len() =>
  math.sqrt(x**2 + y**2) }

// In main.ls:
import .math_utils
let area = PI * r ** 2
let v = <Vec2 x: 3.0, y: 4.0>
v.len() // 5.0
v is Vec2 // true
```

Error Handling

Creating Errors:

```
error("Message")           Message only
error("load failed", inner_err) With an inner error
error({code: 304, message: "div by zero"}) Structured payload
```

Error Return Types (T_E):

```
fn parse(s: string) int^ {...} int or any error
fn divide(a, b) int ^ DivErr    specific error
{...}
fn load(p) Config ^ ParseErr| multi errors
IOErr {...}
```

raise error , or propagate error with ^

```
fn compute(x: int) int^ {
  if (b == 0) raise error("div by zero") // raise error
  let a = parse(input)^ // return immediately on error
  let b = divide(a, x)^ // return immediately on error
  a + b
}
fun()^ // propagate error, discard value
```

e ^ { } — handle the error here (- is the error):

```
pn main() {
  let result = divide(10, x) ^ {
    print(^.message) // ^ = the current handler error
    0 // handler value, or raise/return
  }
  print(result * 2) // result is clean here
}
```

e or default rescues any falsy value without seeing it;
x is error tests a soft T | error value.

Operator Precedence (High to Low)

1. () [] . # ? .? - Primary, force, query
2. - + * & - Unary (*: spread, &: address-of)
3. ** - Exponentiation (right-associative)

4. * / div % - Multiplicative
5. + - ++ - Additive, concatenation
6. < <= > >= - Relational
7. == != === eq ne lt le gt ge - Equality (===: reference equality; the word forms are element-wise)
8. to - Range
9. & - Type intersection
10. ! - Type exclusion
11. | - Type union
12. is in at <: - Type test, value and key membership, subtype (is nan for NaN detection)
13. not - Logical NOT: not a == b is not (a == b)
14. and - Logical AND
15. or - Logical OR
16. |> |: that - Pipe, Filter and Proviso

There is no unary ! in expressions: use not. In type position !T is the complement type, as in x is !null.

Quick Examples

Data Processing:

```
let data = input("sales.json", 'json')^
let total = sum(
  (for (sale in data.sales) sale.amount))
let report = {total: total,
  count: len(data.sales)}
format(report, 'json')
```

Function Definition:

```
fn factorial(n: int) int {
  if (n <= 1) 1 else n * factorial(n - 1)
}

let article = <article title: "My Article",
  <h1 "Introduction">
  <p "Content goes here.">
>
format(article, 'html')
```

Comprehensions - Complex data processing:

```
(let data = [1, 2, 3, 4, 5],
let filtered = (for (x in data)
  if (x % 2 == 0) x else 0),
let doubled = (for (x in filtered) x * 2), doubled)
```